Chapter 1

Markov Decision Processes

Contents

1 Introduction

In this chapter, we discuss **Markov decision process**, a framework that reinforcement learning heavily relys on. Markov decision process is the foundation of reinforcement learning algorithms. We mainly focus on discrete Markov decision process.

The following property of conditional expectation is used often in this document; for example in the derivation of Bellman equations.

$$\mathbb{E}[\mathbb{E}[X|Y,Z]|Y] = \mathbb{E}[X|Z] \tag{1.1}$$

2 Finite Markov Decision Process

2.1 Definition

The learner and decision maker is called *agent*, and the *environment* is the thing the agent interacts with. These two interact at each of a sequence of time step $t = 0, 1, 2, 3, \ldots$. At every time step t, the agent receives *state* $S_t \in \mathcal{S}$ of the environment and selects an action $A_t \in \mathcal{A}(s)$ based on that state s. Next step, the agent receives a reward $R_{t+1} \in \mathcal{R}$ and finds itself in a new state S_{t+1}. The MDP then give rise to the *trajectory* $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots$.

The states, rewards and actions are all have finite number of elements in *finite* MDP. We will mainly discuss about finite MDP.

Definition 2.1.1 (Markov Decision Process (MDP))

Markov decision process is a tuple of five objects $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ where $\mathcal{S}$, $\mathcal{R}$, and $\mathcal{A}$ are sets of states, rewards, and actions. The function $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, 1]$ is called *state-transition probability* and defined as

$$p(s'|s,a) = \mathbb{P}\{S_t = s'|S_{t-1} = s, A_{t-1} = a\} = \sum_{r\in\mathcal{R}} p(s', r|s, a).$$

The function $p(s', r|s, a) = \mathbb{P}\{S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a\}$ is called *dynamics* of the MDP. $\gamma \in [0, 1]$ is called *discount rate*, and is used to calculate the expected value of cummulative rewards, namely the *return* and the expectation of it is what the agent has to maximize. The return is defined as

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

Note that the goal can be written in recursive manner; $G_t = R_{t+1} + \gamma G_{t+1}$.

2.2 Markov Property

The important property of Markov decision process is that R_t and S_t depends only on previous state and action; S_{t-1} and A_{t-1}. This is called **Markov property**. More formally, the Markov property can be defined as following;

Definition 2.2.1 (Markov Property)

Let S_t, R_t and A_t be the state, reward and action at time t. Then, the probability distribution satisfies **Markov property** if

$$P\{S_t = s', R_t = r | S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}, \ldots, S_0 = s_0, A_0 = a_0\} = P\{S_t = s', R_t = r | S_{t-1} = s_{t-1}, A_{t-1} = a_{t-1}\}$$

for all $s', s_i \in \mathcal{S}$, $r \in \mathcal{R}$ and $a_i \in \mathcal{A}$.

3 Episodic Task and Continuing Task

The *return* $G_t = R_{t+1} + \gamma^2 R_{t+2} + R_{t+3} + \ldots$, presented in Definition 2.1.1 shortly, is the value which we should maximize about expectation. The return may consist of finite rewards, or it can be formulated as an infinite sum.

When the agent-environment interaction can break naturally into subsequences, for example winning the chess game and starting it all again from the start, we call these subsequences **episodes**. All the episodes terminate in the special state called **terminal state**. Then, the return can be expressed in finite sum;

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-1} R_T$$

where the time of termination, T is a random variable that normally varies from episode to episode and $0 \leq \gamma \leq 1$ is the discount rate. The states other than teminal state is often written as $\mathcal{S}$, and all the states including terminal state is often written as $\mathcal{S}^+$.

When the agent-environment interaction cannot be expressed in terms of episodes, we call it **continuing task**. The return now cannot be formulated as finite sum;

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{1.2}$$

$$= R_{t+1} + \gamma G_{t+1} \tag{1.3}$$

This equation still holds when the task is episodic; We can set $G_T = 0$ for terminal time T.

The episodic and continuing tasks can be expressed in unified notation by using special *absorbing state* which transitions only to itself and generates only zero rewards. Then the return can be expressed as

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$$

with possibility $T = \infty$ and $\gamma = 0$, but not both.

4 Policies and Value Functions

Now we introduce *value functions* which represents the estimate of *how good* it is for the agent to be in a given state or to perform a given action in a given state. The 'how good' is expressed in terms of expectations of returns.

4.1 Definitions

Definition 4.1.1 (Policy)

The **policy** $\pi(a|s)$ is a mapping from states to probablities of selecting each possible action, where $a \in \mathcal{A}(s)$.

If the agent is following policy $\pi(a|s)$ at time t, the probability of selecting $A_t = a$ in $S_t = s$ is $\pi(a|s)$.

Definition 4.1.2 (Value Function)

The **value function** $v_\pi(s)$ of a state s is the expected return when starting in s and follwing policy π, i.e.,

$$v_\pi(s) = \mathbb{E}_\pi[G_t|S_t = s] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s\right]$$

where $\mathbb{E}_\pi$ denotes the expected value of a random variable given that the agent follows policy π. The function v_π itself is called **state-value function for policy** π.

The value of terminal state is always 0.

Definition 4.1.3 (Action-Value Function)

The **action-value function** $q_\pi(s, a)$ is the expected return starting from state s, taking the action a, and then following policy π.

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t|S_t = s, A_t = a] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s, A_t = a\right]$$

Note that the value function and action-value function has following relationship:

Theorem 4.1.1 (Relationship of Value Function and Action-Value Function)

The state-value fuction $v_\pi(s)$ and action-value function $q_\pi(s, a)$ satisfies following property:

$$v_\pi(s) = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s, a)$$

$$q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1})|S_t = s, A_t = a]$$

Proof. Use the equation 1.1. The second equation can be derived using equation 1.1 with the Markov property. Note that the expectation is taken about policies. □

These functions can be estimated from experience. This type of estimation method is one of Monte Carlo methods. When there are too much states and actions or when they are continuous, we can approximated the function using parametrized approximator, such as deep neural network.

4.2 Bellman Equation

The fundamental property of value functions that are used throughout reinforcement learning and dynamic programming is that they satisfy recursive relationship, called **Bellman equation** for v_π.

Theorem 4.2.1 (Bellman Expectation Equation for v_π)

Let v_π be a state-value function for policy π. Then the v_π satisfies following equation;

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi[G_t|S_t = s] && (1.4)\\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s] && (1.5)\\
&= \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'} \sum_{r} p(s', r|s, a)\big[r + \gamma \mathbb{E}_\pi[G_{t+1}|S_{t+1} = s']\big] && (1.6)\\
&= \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma v_\pi(s')], \text{ for all } s \in \mathcal{S} && (1.7)
\end{aligned}
$$

which is called the **Bellman equation** for v_π.

The actions a are taken from $\mathcal{A}(s)$. The sum can be understood as expectation, since it is a sum over all three variables a, s, and r. It is just a sum of returns $r + \gamma v_\pi(s')$ multiplied by the probability $\pi(a|s)p(s', r|s, a)$.

The following is the Bellman equation for action-value function.

Theorem 4.2.2 (Bellman Expectation Equation for q_π)

Let q_π be an action-value function for policy π. Then q_π satisfies following equation;

$$
\begin{aligned}
q_\pi(s, a) &= \mathbb{E}_\pi[G_t|S_t = s, A_t = a] && (1.8)\\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1}|S_t = s, A_t = a] && (1.9)\\
&= \sum_{s',r} p(s', r|s, a)\big[r + \gamma \mathbb{E}_\pi[G_{t+1}|S_{t+1} = s']\big] && (1.10)\\
&= \sum_{s',r} p(s', r|s, a)\big[r + \gamma v_\pi(s')\big] && (1.11)\\
&= \sum_{s',r} p(s', r|s, a)\left[r + \gamma \sum_{a'\in\mathcal{A}(s')} \pi(a'|s') q_\pi(s', a')\right], \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s) && (1.12)
\end{aligned}
$$

which is called the **Bellman equation** for q_π.

Note that we used law of total expectation; check 1.1.

5 Optimal Policies and Optimal Value Functions

5.1 Definition

Value functions define a partial ordering over policies. We denote $\pi \geq \pi'$ if and only if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$. The **optimal policy** is a policy that is better than or equal to all other policies. There is always at least one optimal policy. There can be more than one optimal policies, but they all share the same state-value function, namely the *optimal state-value function* and the same *optimal action-value function*.

Definition 5.1.1 (Optimal State-Value Function)

Let v_π be a state-value function. Then, the **optimal state-value function** v_* is defined as

$$v_*(s) = \max_\pi v_\pi(s)$$

for all $s \in \mathcal{S}$.

Definition 5.1.2 (Optimal Action-Value Function)

Let q_π be a action-value function. Then, the **optimal action-value function** q_* is defined as

$$q_*(a, s) = \max_\pi q_\pi(a, s)$$

for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

5.2 Bellman Optimality Equation

The optimal functions presented above also satisfies the Bellman equation, called the **Bellman optimality equation**;

Theorem 5.2.1 (Bellman Optimality Equation)

Let v_* and q_* be optimal state-value function and optimal action-value function. Then, these functions satisfy the following **Bellman optimality equation**;

$$\begin{aligned} v_*(s) &= \max_{a \in \mathcal{A}} q_*(s, a) && (1.13)\\ &= \max_a \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a] && (1.14)\\ &= \max_a \sum_{s', r} p(s', r|s, a)[r + \gamma v_*(s')] && (1.15)\\ q_*(s, a) &= \mathbb{E}[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a] && (1.16)\\ &= \sum_{s', r} p(s', r|s, a)[r + \gamma \max_{a'} q_*(s', a')] && (1.17) \end{aligned}$$

Note that we used following relationship between optimal functions;

Theorem 5.2.2 (Relationship Between Optimal Functions)

Let v_* and q_* be optimal state-value function and optimal action-value function. Then, these functions satisfy the following relationship;

$$v_*(s) = \max_{a \in \mathcal{A}(s)} q_*(s, a) \tag{1.18}$$

Moreover, following holds for optimal policy π_*;

$$\pi_*(s) = \arg \max_{a \in \mathcal{A}(s)} q_*(s, a) \tag{1.19}$$

Proof. The proof for first equation uses $a = b \Leftrightarrow a \leq b$ and $b \leq a$. The second equation can be proved using the first equation. □

Once one has v_*, determining an optimal policy is straightforward. We just have to make one step along, and then find out the actions that satisfy the Bellman optimality equation. We then create the policy that allocates positive probability only to these actions. This *greedy* selection seems that it ignores possibility of better alternatives in the long run. However, note that we are trying to maximize the *return*, which condsiders a long-run cummulative sum, and the optimal functions are derived using this return. Thus, we do not have to worry about the better alternatives.

Having q_*, it is even more strightforward to create an optimal policy. We don't even have to take a step ahead; since we can always find optimal action a, for any given state s.